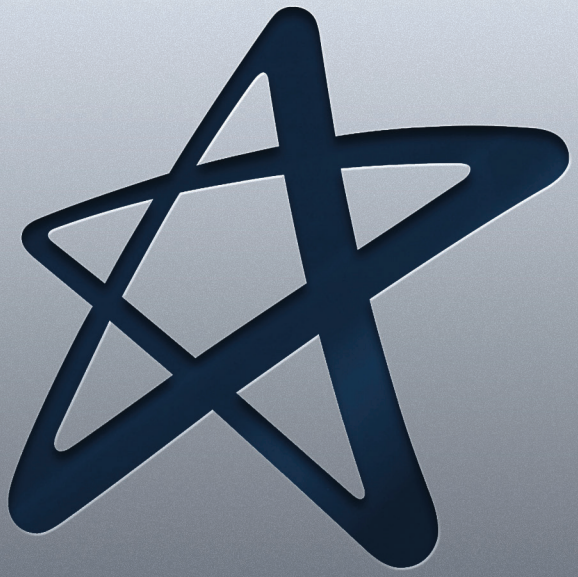
Linguagens Formais e Autômatos



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico

Linguagens Regulares: Autômatos

Responsável pelo Conteúdo:
Prof. Dr. Cleber Silva Ferreira da Luz

Revisão Textual:
Prof. Me. Luciano Vieira Francisco

UNIDADE Linguagens Regulares: Autômatos

- Introdução;
- Sistema de Estados Finitos;
- Autômato.



OBJETIVO DE APRENDIZADO

- Conhecer os conceitos básicos sobre autômatos.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu "momento do estudo";
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Na teoria das linguagens formais, uma linguagem formal pode ser vista como mecanismos formais para representação e especificação de linguagens. Habitualmente, as representações são realizadas por reconhecedores e geradores. Os reconhecedores são mecanismos formais utilizados para verificar se uma palavra pertence ou não a uma linguagem. Assim, nesta Unidade estudaremos os reconhecedores autômatos (RAMOS; VEGA; JOSÉ NETO, 2009).

Um autômato é um formalismo matemático reconhecedor de linguagens. Sendo composto por estados e transações, um autômato reconhece se um pertence a um alfabeto; logo, um autômato pode ser considerado com um sistema de estados finitos. A próxima seção apresenta mais sobre sistema de estados finitos.

Sistema de Estados Finitos

Um sistema de estados finitos pode ser considerado como um modelo matemático de sistema com entradas e saídas discretas. Tal modelo pode assumir um número finito e predefinido de estados, de modo que cada estado guarde apenas as informações do passado necessárias para determinar as ações para a próxima entrada (MENEZES, 2011).

Para facilitar o entendimento de um sistema de estados finitos, considere um elevador: trata-se de um sistema que não memoriza as requisições anteriores. Logo, os andares são aqui representados por estados que guardam as informações **andar atual e direção de movimentação**. As entradas ao sistema são as denominadas *pendentes*. Assim como um elevador, um sistema de estados finitos também é formado por estados – andares – que armazenam somente as informações *onde se está* e para *aonde vai* dada uma entrada (MENEZES, 2011).

Podemos citar como outros exemplos de aplicações de sistemas de estados finitos os processadores de textos e os analisadores léxicos – em ambos os casos, basicamente é memorizada a estrutura do prefixo da palavra em análise, apenas.

Agora que entendemos perfeitamente o que é um sistema de estados finitos, podemos compreender com facilidade o que é um autômato, de modo que a próxima seção o explicará com detalhes.

Autômato

Um autômato pode ser visto como um sistema de estado finito onde é formado um modelo computacional sequencial que pode ser aplicado em diversas áreas da computação, tais como linguagens formais, compiladores, semântica formal e modelos para concorrência (MENEZES, 2011; SIPSER, 2007).

Um autômato é considerado um formalismo operacional ou reconhecedor, que pode ser:

- **Determinístico:** dependendo do símbolo lido e do estado corrente – atual –, o sistema pode assumir um único estado bem definido;
- **Não determinístico:** dependendo do símbolo lido e do estado corrente – atual –, o sistema pode assumir um conjunto de estados alternativos;
- **Com movimento vazio:** dependendo do estado atual e sem ler símbolo algum, o sistema pode assumir um conjunto de estados.

Em autômatos determinísticos, tal como o próprio nome induz, a transição de um estado ao outro é determinística, isto é, a transição é certeira. Em contrapartida, em autômatos não determinísticos, a transição pode assumir um conjunto de estados, isto é, a transição não é certeira – dado um conjunto de estados possíveis, o sistema pode assumir qualquer um.

Em autômatos com movimentos vazios, a transição pode assumir um conjunto de estados, logo, autômatos com movimentos vazios também não são determinísticos. Você deve estar se perguntando: "mas qual é a vantagem de estar em um estado, e sem ler qualquer símbolo de entrada, o sistema assumir outra entrada? Qual é a vantagem disto?" Pois bem, vamos lá: movimentos vazios podem ser entendidos como movimentos encapsulados, nos quais, excetuando-se por uma eventual mudança de estado, de modo que nada mais pode ser observado; de forma análoga, podemos citar o encapsulamento das linguagens orientadas a objetos.

Em termos computacionais, os três tipos de autômatos possuem o mesmo poder computacional. Considerando que aqui autômatos são sistemas de estados finitos, autômatos determinísticos também são chamados de **Autômatos Finitos Determinísticos** (AFD); enquanto autômatos não determinísticos são denominados **Autômatos Finitos Não Determinísticos** (AFN) (MENEZES, 2011).

Um AFD pode ser compreendido como uma máquina formada, basicamente, por três partes:

- **Fita:** que pode ser vista como um dispositivo de entrada que contém a informação a ser processada;
- **Unidade de controle:** que reflete o estado corrente da máquina. Este possui uma unidade de leitura – cabeça da fita –, a qual acessa uma célula da fita de cada vez e movimenta-se somente para a direita;
- **Programa, função programa, ou função de transição:** esta parte é responsável por comandar as leituras e definir o estado da máquina.

Em relação à fita, podemos observar que é finita, dividida em células, sendo que cada célula armazena um símbolo; tais símbolos pertencem a um alfabeto de entrada. Não se pode gravar sobre a fita, de modo que a palavra que será processada, inicialmente, ocupará toda a fita.

Sobre a unidade de controle, podemos observar que possui um número finito e predefinido de estados, originando, assim, o termo **controle finito**. A unidade de controle processa a fita, lendo os símbolos de uma célula de cada vez. A cada leitura, a cabeça da fita move uma célula para a direita. Para iniciar o processamento, a cabeça fica posicionada na célula mais à esquerda da fita (Figura 1).

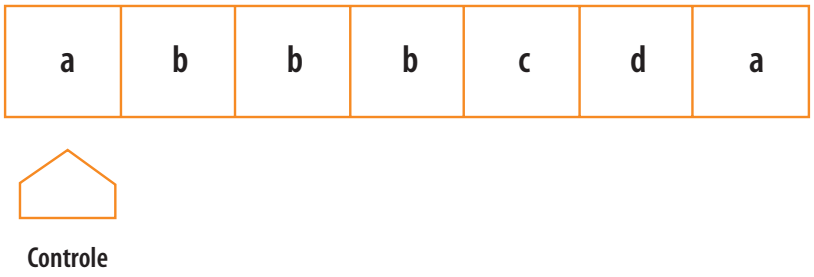


Figura 1 – Autômato finito como uma máquina com controle finito

O programa é uma função parcial que, dependendo do estado corrente e do símbolo lido, determina o novo estado do autômato.

Autômato Finito Determinístico

Formalmente, um AFD é uma 5-upla ordenada: $M = (\Sigma, Q, \delta, q_0, F)$ onde:

- Σ é o alfabeto de entrada;
- Q é o conjunto finito de estados possíveis do autômato;
- δ é uma função programa, igualmente chamada de **função transição**. Por exemplo, suponhamos que a função programa é definida para um estado p e um símbolo a, resultando no estado q, então, teremos: $\delta(p,a) = q$. A função programa é responsável pela transição de estados nos autômatos. Nesse exemplo, o sistema estava no estado p, de modo que ao ler um símbolo a passou ao estado q;
- q_0 é um elemento distinguido de Q, chamado de **estado inicial**;
- F é um subconjunto de Q, chamado, no coletivo, de **estados finais**.

Além da 5-upla, um autômato pode ser representado na forma de diagrama, conforme a seguir:

- Estados são nós, representados por círculos;
- As transições são representadas por arestas que ligam os nós correspondentes (Figura 2a);
- Os estados inicial e final são representados diferentemente dos demais estados (Figura 2d);
- Podem ocorrer transições paralelas (Figura 2b). A transação que ocorre na Figura 2c é equivalente à que acontece na Figura 2b.

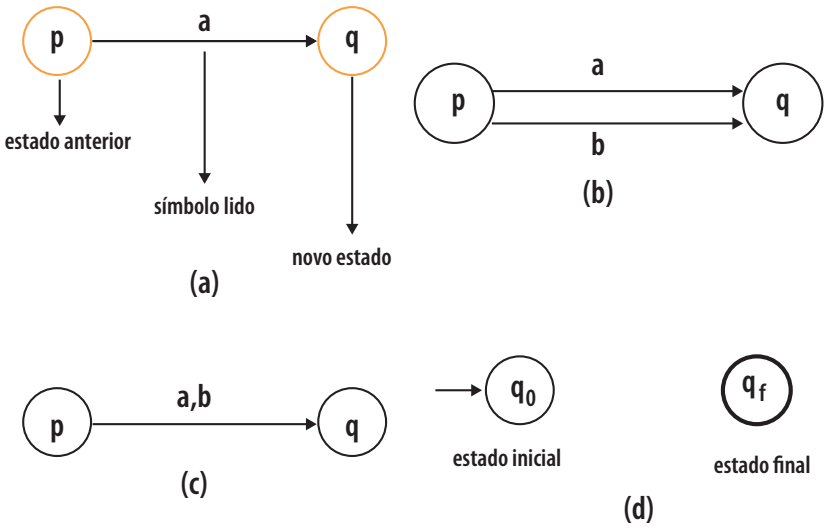


Figura 2 – Exemplos de AFD
Fonte: Adaptada de Menezes (2011)

Uma função programa também pode ser representada por uma tabela de dupla entrada. Por exemplo, a transição do tipo $\delta(q_1,a)q_2$ pode ser representada em forma de tabela:

Tabela 1

Δ	A	...
q_1	q_2	...
q_2	...	...

Computação de Autômato Finito

Para uma palavra *w* dada como entrada, a computação de um autômato consiste na **sucessiva aplicação da função programa** para cada **símbolo de *w***, sempre da esquerda para a direita, até ocorrer uma condição de parada (MENEZES, 2011).

O seguinte exemplo consiste em um autômato para *aa* ou *bb* como subpalavras. Para este caso será considerado o alfabeto $\Sigma = \{a, b\}$: $L_1 = \{w \mid w$ possui *aa* ou *bb* como subpalavra$\}$:

Eis o autômato finito:

$$M_1 = (\{a,b\}, \{q_0, q_1, q_2, q_f\}, \delta 1, q_0, \{q_f\})$$

Onde a função programa δ_1 é dada pela seguinte Tabela:

Tabela 2

Δ	A	b
q_0	q_1	q_2
q_1	q_f	q_2
q_2	q_1	q_f
q_f	q_f	q_f

Enquanto o autômato M_1 é representado pelo seguinte diagrama:

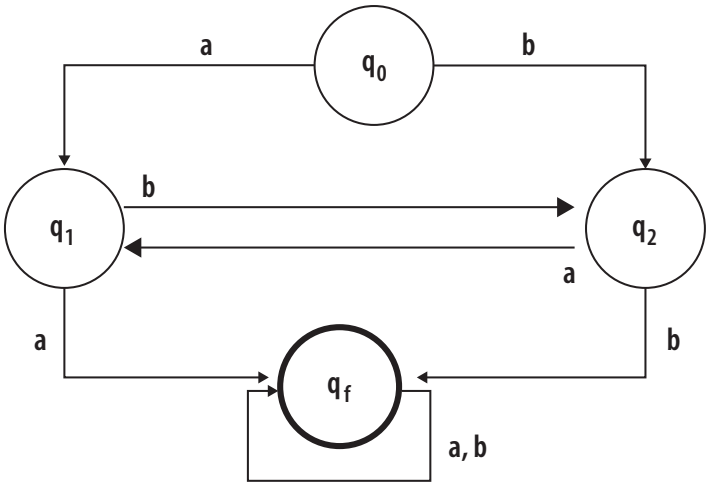


Figura 3 – Autômato finito determinístico
Fonte: Adaptada de Menezes (2011)

Neste autômato, após identificar dois *a* ou dois *bb* consecutivos é assumido o estado q_f (final) e varrido o sufixo da palavra de entrada, somente para terminar o processamento – a Figura 4 ilustra a computação do autômato finito M_1 para a entrada w = abba, que é aceita pelo autômato:

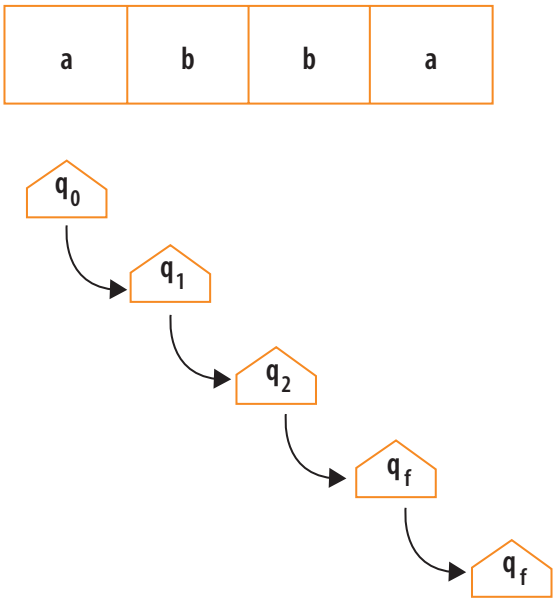


Figura 4 – Computação (AFD): sequência de dois símbolos iguais
Fonte: Adaptada de Menezes (2011)

Um autômato finito sempre para ao processar qualquer entrada *w*, afinal, como qualquer palavra é finita e como um novo símbolo da entrada é lido a cada aplicação da função programa, não existe a possibilidade de ciclo – laço – infinito (MENEZES, 2011).

Assim, no processamento de uma palavra w, a parada de um autômato finito pode ser de duas maneiras:

- Aceita a entrada w, após processar o último símbolo da fita, de modo que o autômato finito assume um estado final;
- Rejeita a entrada w, ocorrendo uma das duas seguintes possibilidades:
  - Após processar o último símbolo da fita, o autômato finito assume um estado não final;
  - Em algum momento, ao longo do processamento de w, a função programa é indefinida ao argumento – estado corrente e símbolo lido da fita.

Vale ressaltar que para verificar se um autômato finito **aceita** ou **rejeita** uma entrada w, deve-se aplicar a função programa para a entrada w a partir do estado inicial do autômato.

Linguagem Aceita, Linguagem Rejeitada

Seja $M = (\Sigma, Q, \delta, q_0, F)$ um AFD, a linguagem reconhecida por M será denotada por:

Aceita(M) ou $L(M)$

É o conjunto de todas as palavras pertencentes a Σ^* aceitas por M, iniciando pelo estado q_0, isto é:

$$L(M) = \text{Aceita}(M) = \{w \mid \delta^*(q_0,w) \in F\}$$

Já a linguagem rejeitada por M é denotada por:

Rejeita(M)

É o conjunto de todas as palavras que pertencem a Σ^* rejeitadas por M, a partir do estado q_0, isto é:

$$\text{Rejeita}(M) = \{w \mid \delta^*(q_0, w) \notin F \text{ ou } \delta^*(q_0,w) \text{ é indefinida}\}$$

Suponhamos que Σ^* é o conjunto universo, de modo que as seguintes afirmações se apresentem como verdadeiras:

- $\text{Aceita}(M) \cap \text{Rejeita}(M) = \varnothing$;
- $\text{Aceita}(M) \cup \text{Rejeita}(M) = \Sigma^*$;
- $\sim\text{Aceita}(M) = \text{Rejeita}(M)$;
- $\sim\text{Rejeita}(M) = \text{Aceita}(M)$;

Dessa forma, cada autômato finito M definido sobre o alfabeto Σ induz uma partição do conjunto de todas as palavras Σ^* em duas classes de equivalência:

Aceita(M) e Rejeita(M). Isto é ilustrado na Figura 5:

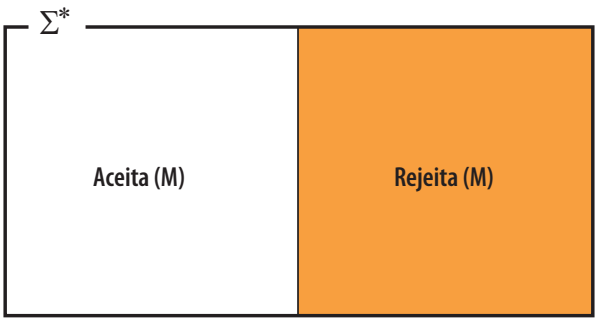


Figura 5 – Partição de Σ*
Fonte: Adaptada de Menezes (2011)

Em palavras simples, para saber se uma palavra é aceita ou não por um AFN, basta simular o processamento dessa palavra pelo AFD. Caso ao processar toda a palavra e o último símbolo dessa palavra cair em um estado final, a palavra será aceita pelo AFD. Contudo, se ao terminar de processar a palavra e não atingir um estado final, ou em determinado momento do processamento não se saber qual estado assumir, a palavra não será aceita (MENEZES, 2011).

Autômatos Finitos Equivalentes

Dois autômatos finitos M_1 e M_2 são ditos equivalentes se, e somente se:

Aceita(M_1) = Aceita(M_2)

Linguagem Regular, Linguagem Tipo 3

Na teoria das linguagens formais, uma linguagem L é dita linguagem regular, ou linguagem de tipo 3, se existir, pelo menos, um AFD que aceite L.

Exemplo 1 – autômato finito (vazia, todas as palavras)

Considere as seguintes linguagens sobre o alfabeto {a, b}

$L_2 = \varnothing$ e $L_3 = \Sigma^*$

E os autômatos finitos:

$M_2 = (\{a,b\}, \{q_0\}, \delta_2, q_0, \{\})$ e $M_3 = (\{a,b\}, \{q_0\}, \delta_3, q_0, \{q_0\})$

Estes dois autômatos são ilustrados na Figura 6, de modo que as funções programas dos quais são representadas na Figura 7:

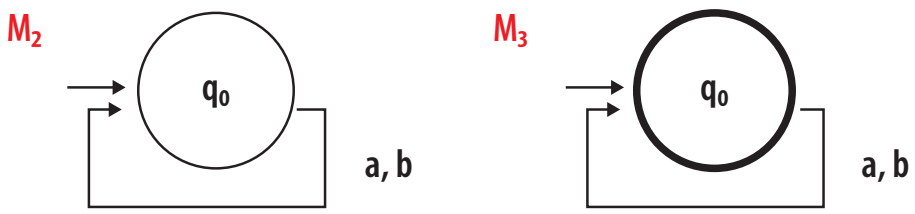


Figura 6 – Partição de Σ* – Legenda: AFD vazio (esquerda) e AFD todas as palavras (direita)
Fonte: Adaptada de Menezes (2011)

Tabela 3 – Função programa vazio (em cima) e função programa de todas as palavras (em baixo)

δ_2	a	b
q_0	q_0	q_0
δ_3	a	b
q_0	q_0	q_0

Logo:

Aceita(M_2) = L_2 e Aceita(M_3) = L_3.

Portanto, as linguagens são regulares.

Exemplo 2 – autômato finito (número par de cada símbolo)

Considere a seguinte linguagem sobre o alfabeto $\{a, b\}$:

$L_4 = \{w \mid w$ possui um número par de a e um número par de $b\}$

O autômato finito:

$$M_4 = (\{a, b\}, \{q_0, q_1, q_2, q_3\}, \delta_4, q_0, \{q_0\})$$

Este autômato é ilustrado na Figura 8:

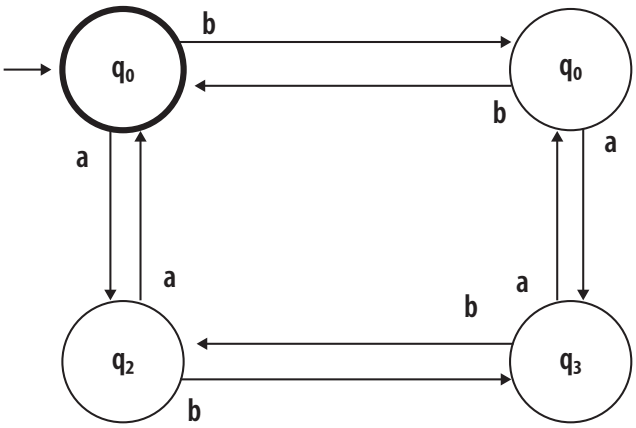


Figura 7 – Diagrama AFD: número par de cada símbolo
Fonte: Adaptada de Menezes (2011)

Este autômato é tal que:

Aceita(M_4) = L_4.

Dessa forma, L_4 é uma linguagem regular.

Autômato Finito Não Determinístico

AFN é uma variação importante de autômatos finitos, dado que nesta variação a transição de estado possui um conjunto de novos estados que poderão ser assumidos, dependendo do símbolo lido; isto é, o próximo estado a ser escolhido é uma opção entre os diversos estados alternativos – a Figura 9 exemplifica um autômato finito não determinístico:

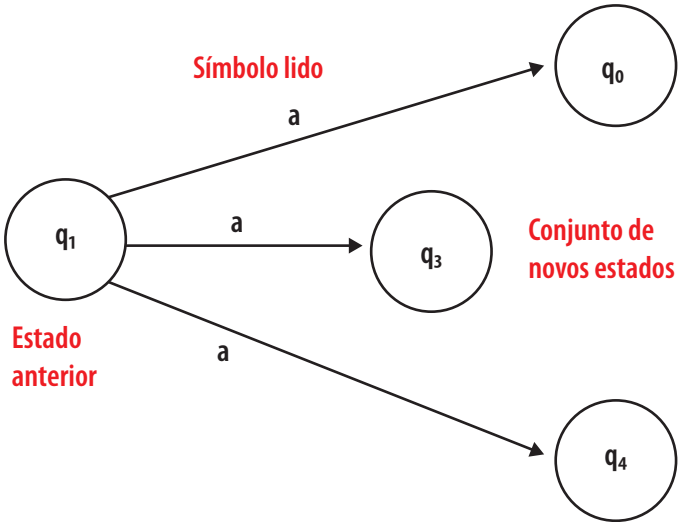


Figura 8 – Autômato finito não determinístico
Fonte: Acervo do conteudista

Note que neste autômato, estando no estado q_1 e lendo o símbolo a, o sistema pode assumir os estados q_2, q_3 ou q_4, tornando a transição de estado não determinística (MENEZES, 2011).

Ademais, um AFN pode ser classificado como:

- **Interno:** o sistema escolhe aleatoriamente o próximo estado a ser executado;
- **Externo:** a escolha do próximo estado a ser executado é externa ao sistema.

Sendo composto por uma fita, unidade de controle e programa, um AFN pode assumir um conjunto de estados alternativos, tal como se houvesse multiplicação de unidades de controle – uma para cada alternativa –, processando independentemente, ou seja, sem compartilhar recursos com outrem. Dessa forma, o processamento de um caminho não influi no estado, símbolo lido e posição da cabeça dos demais caminhos alternativos.

Assim como o AFD, AFN é uma 5-upla ordenada, vejamos:

$$M = (\Sigma, Q, \delta, q_0, F)$$

- **Onde:**
  - Σ é o alfabeto de entrada;
  - Q é o conjunto finito de estados possíveis do autômato;
  - δ é uma função programa, igualmente chamada de **função transição**;
  - q_0 é um elemento diferenciado de Q, chamado de **estado inicial**;
  - F é um subconjunto de Q, chamado, coletivamente, de **estados finais**.

Os componentes Σ, Q, δ q_0, F do AFN são os mesmos do AFD. A representação por diagrama do AFN também é semelhante à do AFD; a única diferença é que o AFN possui mais que um caminho para uma transição de estado, enquanto que no

AFN a representação da função programa é análoga à do AFD. Trata-se de uma transição do tipo:

$$\delta\ (p,\ a) = \{q_1,\ q_2 \ldots\ q_n\}$$

Resultando em diversas arestas etiquetadas por *a*, com origem em *p* e com destino em cada estado q_1, $q_2 \ldots\ q_n$.

No AFN, a computação de uma palavra *w* consiste na sucessiva aplicação da função programa para cada símbolo de *w*, da esquerda à direita, até ocorrer uma condição de parada, assim como no AFD.

Igualmente como o AFD, a parada de processamento de uma AFN para uma entrada *w* pode se dar de duas formas:

- **Aceita** a entrada *w*, após processar o último símbolo da fita, existindo, pelo menos, um estado final pertencente ao conjunto de estados alternativos atingidos;
- **Rejeita** a entrada *w*, desdobrando-se em duas possibilidades:
  - Após processar o último símbolo da fita, todos os estados alternativos atingidos são não finais;
  - Em algum momento, ao longo do processamento de *w*, a função programa é indefinida ao argumento – conjunto de estados alternativos e símbolo lido da fita.

Seja $M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$ um AFN, a linguagem aceita ou reconhecida por *M* é denotada por:

Aceita(*M*) ou L(*M*)

Analogamente, a linguagem rejeitada ou não reconhecida por *M* é denotada por:

Rejeitada(*M*)

Logo, para saber se uma palavra é aceita ou não por um AFN, basta simular o processamento da palavra pelo AFN: caso ao processar toda a palavra e o último símbolo cair em um estado final, a palavra será aceita pelo AFN; contudo, caso ao terminar de processar a palavra e não atingir um estado final, ou em determinado momento do processamento não se saber o estado a assumir, a palavra não será aceita (MENEZES, 2011).

Nesse sentido, analisemos dois exemplos de AFN:

Exemplo 3 – autômato finito não determinístico (aa ou bb como subpalavra)

Considere a seguinte linguagem sobre o alfabeto {*a*, *b*}:

$L_5 = \{w \mid w$ possui *aa* ou *bb* como subpalavra$\}$

O autômato finito não determinístico:

$$M_5 = (\{a,\ b\},\ \{q_0,\ q_1,\ q_2,\ q_f\},\ \delta 5,\ \{q_f\})$$

A função programa δ_5 é dada pela seguinte Tabela:

Tabela 4

δ	*a*	*b*
q_0	$\{q_0, q_1\}$	$\{q_0, q_2\}$
q_1	q_f	-
q_2	-	q_f
q_f	q_f	q_f

Fonte: Elaborado pelo conteudista

Já M_5 é representado pelo seguinte diagrama:

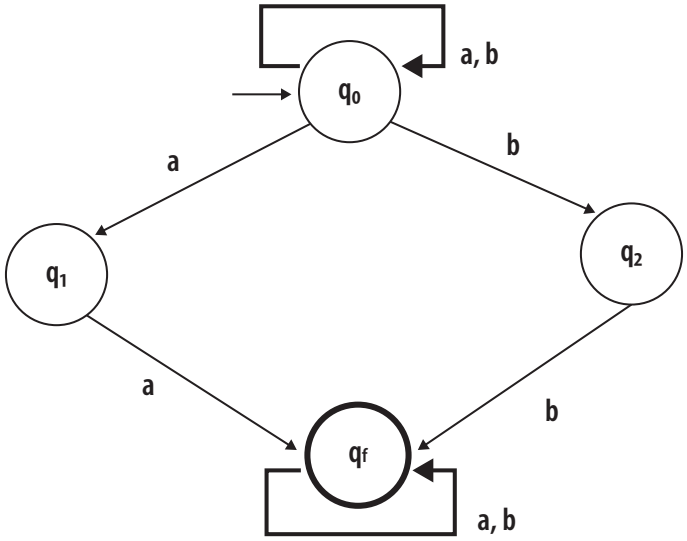


Figura 9 – Autômato finito não determinístico (Exemplo 3)
Fonte: adaptada de Menezes (2011)

Como é possível observar na Figura 10, o autômato finito apresentado possui um conjunto de estados alternativos. Estando no estado inicial q_0 e lendo um *a*, é possível ficar em q_0 ou ir para q_1. Igualmente, estando no estado inicial q_0 e lendo um *b*, é possível ficar em q_0 ou ir para q_2 – tais alternativas de caminhos caracterizam AFN.

Um algoritmo de processamento pode ser dado pela varredura sobre a palavra *w* de entrada. A cada ocorrência de *a* – e respectivamente de *b* – uma alternativa é iniciada para verificar se o símbolo seguinte também é *a* – e respectivamente *b*. Dessa forma, existem três caminhos alternativos de processamento, onde o:

- Ciclo em q_0 realiza uma varredura em toda a entrada;
- Caminho $q_0/q_1/q_f$ garante a ocorrência de *aa*;
- Caminho $q_0/q_2/q_f$ garante a ocorrência de *bb*.

Exemplo 4 – autômato finito não determinístico (aaa como sufixo)

Considere a seguinte linguagem sobre o alfabeto (*a*, *b*):

$L_6 = \{w \mid w$ possui aaa como sufixo$\}$

O autômato finito não determinístico:

$$M_6 = (\{a, b\}\ \{q_0, q_1, q_2, q_f\}, \delta_6, q_0, \{q_f\})$$

E M_6 ilustrado pelo seguinte diagrama:

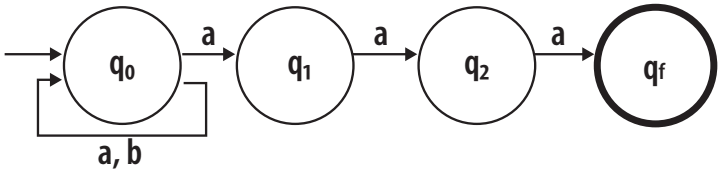


Figura 10 – Autômato finito não determinístico (Exemplo 4)
Fonte: Adaptada de Menezes (2011)

Autômato Finito com Movimentos Vazios

Habitualmente, autômatos finitos com movimentos vazios são não determinísticos. Um movimento vazio pode ser considerado como uma transição sem a leitura de símbolo algum. O movimento vazio pode ser interpretado como um não determinístico interno ao autômato, formando uma transação encapsulada – isto é, excetuando-se por uma eventual mudança de estados, nada mais pode ser observado sobre movimentos vazios (MENEZES, 2011).

Assim como AFD e AFN, um autômato finito com movimento vazio é uma 5-upla ordenada, veja:

$$M = (\Sigma, Q, \delta, q_0, F)$$

- **Onde:**
  - Σ é o alfabeto de entrada;
  - Q é o conjunto finito de estados possíveis do autômato;
  - δ é uma função programa, igualmente chamada de **função transição**, que suporta transições como esta: $\delta(p, \varepsilon) = \{q_1, q_2 \ldots q_n\}$
  - q_0 é um elemento diferenciado de Q, chamado de **estado inicial.**

Os componentes Σ, Q, δ q_0, F do autômato com movimento vazio são os mesmos de AFD e AFN – a Figura 12 ilustra um autômato com movimento vazio:

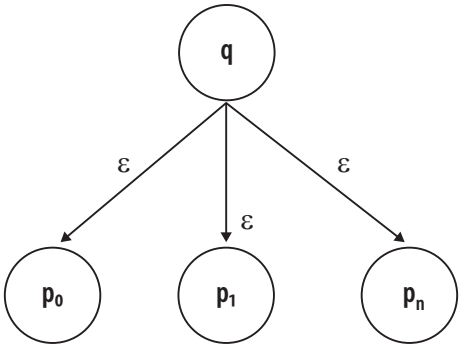


Figura 11 – Autômato finito com movimento vazio
Fonte: elaborada pelo professor conteudista

A computação de um autômato finito com movimento vazio é semelhante à de um autômato finito não determinístico. Adicionalmente, o processamento de uma transição para uma entrada vazia também é não determinístico. Ao processar uma entrada vazia, um AFNε assume simultaneamente os estados destino e origem. Assim, a origem de um movimento vazio sempre é "um caminho alternativo".

Exemplo 5 – autômato com movimentos vazios (a's antecedem b's)

Considere a seguinte linguagem sobre o alfabeto $\{a,b\}$:

$L_7 = \{w \mid$ qualquer símbolo a antecede qualquer símbolo $b\}$

O autômato finito com movimentos vazios:

$$M_7 = (\{a, b\}\ \{q_0, q_f\}, \delta_7, q_0, \{q_f\})$$

E a função programa δ_7 dada pela seguinte Tabela:

Tabela 5

δ	a	b	ε
q_0	$\{q_0\}$	-	$\{q_f\}$
q_f	-	q_f	

Fonte: Elaborado pelo conteudista

M_7 é assim ilustrado:

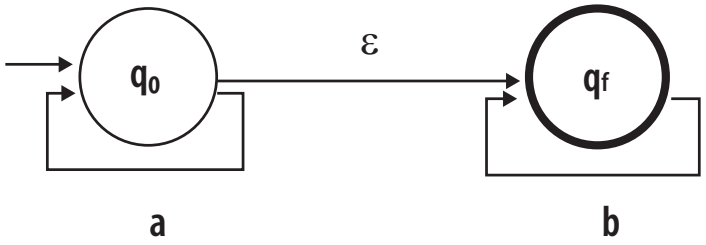


Figura 12 – Autômato finito com movimento vazio
Fonte: adaptada de Menezes (2011)

Equivalência Entre AFD e AFN

A classe dos AFD é equivalente à classe dos AFN.

Prova por Indução

Esta prova consiste em mostrar que, a partir de um AFN M qualquer, torna-se possível construir um AFD M_D que realize as mesmas computações, isto é, M_D simula M. A ideia central do algoritmo é a construção de estados de MD que simulem as diversas combinações de estados alternativos de M.

Assim, seja M = (Σ, Q, δ, q_0, F) um AFN qualquer; seja também: M_D = (Σ, Q_D, δ_D, q_0, F_D) um AFD construído a partir de M – tal como é dado a seguir:

- Q_D é o conjunto criado a partir de todas as combinações, sem repetições, de estados de Q. Cada estado de QD é denotado por: $\{q_1, q_2, ..., q_n\}$
- Onde q_i pertence a Q, para todo i em $\{1, 2... n\}$. Assim, um estado de M_D representa uma imagem de todos os estados alternativos de M;
- A função programa δ_D: $Q_D \times \Sigma \rightarrow Q_D$ é tal que: $\delta_D(\{q_1... q_n\}, a) = \{p_1... p_m\}$ se, e somente se, $\delta^*(\{q_1... q_n\}, a) = \{p_1... p_m\}$
- (q_0) é o estado inicial;
- F_D é o conjunto de todos os estados $(q_1, q_2... q_n)$ pertencentes a Q_D tal que algum estado q_i pertence a F, para todo i em $\{1, 2... n\}$

Agora, a demonstração de que o AFD M_D, de fato, simula exatamente as computações do AFN M é por indução no tamanho da palavra w. É possível mostrar que: $\delta_D^*(\{q_0\}, w) = \{q_1... p_u\}$ se, e somente se, $\delta^*(\{q_0\}, w) = \{q_1... p_u\}$

- Base de indução, seja w tal que $|w| = 0$. Portanto, $w = \varepsilon$: $\delta_D^*(\{q_0\}, \varepsilon) = \{q_0\}$ se, e somente, se $\delta^*(\{q_0\}, \varepsilon) = \{q_0\}$
- Hipótese de indução, seja w tal que $|w| = n$ e $n \geq 1$. Suponhamos que seja verdade que: $\delta_D^*(\{q_0\}, w) = \{q_0... q_u\}$ se, e somente se $\delta^*(\{q_0\}, w) = \{q_0... q_u\}$
- Passo de indução, seja w tal que $|wa| = n + 1$ e $n \geq 1$. Então: $\delta_D^*(\{q_0\}, wa) = \{p_1... p_v\}$ se, e somente se $\delta^*(\{q_0\}, wa) = \{p_1... p_v\}$

O que equivale, por indução: $\delta_D(\{q_1... q_u\}, a) = \{p_1... p_v\}$ se, e somente se $\delta^*(\{q_1... q_u\}, a) = \{p_1... p_v\}$

Logo, M_D simula M para qualquer entrada w pertencente a Σ^*.

Uma linguagem aceita por um autômato finito não determinístico é uma linguagem regular.

Determinístico *Versus* Não Determinístico

Às vezes, é mais fácil desenvolver um autômato finito não determinístico do que um determinístico. Por exemplo, considere um autômato finito não determinístico que aceita a seguinte linguagem sobre $\Sigma = \{a, b\}$:

$\{w \mid$ o quinto símbolo da direita para a esquerda de w é $a\}$

A solução determinística não é trivial e resulta em grande número de estados. Todavia, uma solução não determinística é bem mais simples e precisa de poucos estados. Comumente, para construir um autômato finito determinístico é preferível desenvolver, inicialmente, uma solução não determinística e sobre esta aplicar a equivalência de AFD e AFN.

Exemplo 6 – Construção de AFD a partir de um AFN

Considere o AFN $M_6 = (\{a, b\} \{q_0, q_1, q_2, q_f\}, \delta_6, q_0, \{q_f\})$, representado pelo seguinte diagrama:

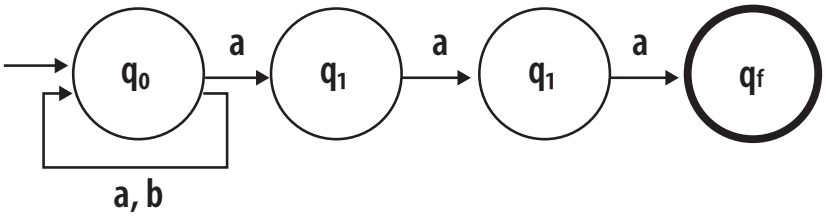


Figura 13
Fonte: adaptada de Menezes (2011)

Como já apresentado, este AFN reconhece a palavra que tenha como sufixo aaa. Assim, o AFD correspondente é: $M_{6D} = (\{a, b\} Q_D, \delta_{6D}, q_0, \{F_D\})$

De modo que a construção fica desta forma:

$Q_D = \{(q_0), (q_1), (q_2), (q_f), (q_0q_1), (q_0q_2)... (q_0q_1q_2q_f)\}$

$F_D = \{(q_f), (q_0q_f), (q_1q_f)... (q_0q_1q_2q_f)\}$

E a função programa δ_6 é dada pela tabela a seguir:

Tabela 6

δ	*a*	*b*
q_0	$\{q_0q_1\}$	$\{q_0\}$
$\{q_0q_1\}$	$\{q_0q_1q_2\}$	$\{q_0\}$
$\{q_0q_1q_2\}$	$\{q_0q_1q_2q_f\}$	$\{q_0\}$
$\{q_0q_1q_2q_f\}$	$\{q_0q_1q_2q_f\}$	$\{q_0\}$

O AFD construído a partir do AFN é apresentado a seguir:

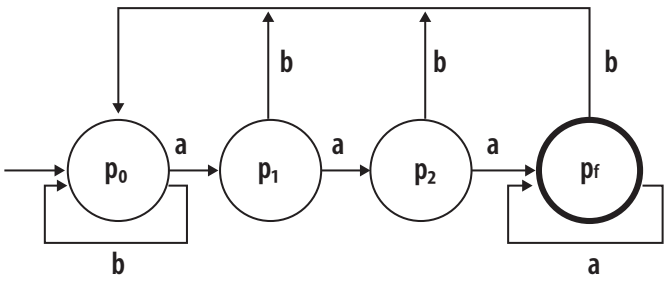


Figura 14
Fonte: adaptada de Menezes (2011)

Por simplicidade, os estados $\{q_0\}$, $\{q_0q_1\}$, $\{q_0q_1q_2q_f\}$ e $\{q_0q_1q_2q_f\}$ foram renomeados por $p_{0,}\, p_{1,}\, p_{2\,e}\, p_f$.

Em Síntese

Autômatos são importantes mecanismos de reconhecimento de palavras, então utilizados para verificar se uma **palavra pertence ou não a uma linguagem**. Nesta Unidade, portanto, abordamos os principais conceitos sobre autômatos.

Um autômato é um formalismo matemático reconhecedor de linguagens, composto por estados e transições; podendo ser:

- **Determinístico:** dependendo do símbolo lido e estado corrente – atual –, o sistema pode assumir um único estado bem definido;
- **Não determinístico:** dependendo do símbolo lido e estado corrente – atual –, o sistema pode assumir um conjunto de estados alternativos;
- **Com movimento vazio:** dependendo do estado atual e sem ler símbolo algum, o sistema pode assumir um conjunto de estados.

Em autômatos determinísticos, a transição de um estado a outro é determinística, isto é, a transição é certeira. Contudo, em autômatos não determinísticos, a transição pode assumir um conjunto de estados, isto é, a transição não é certeira. Dado um conjunto de estados possíveis, o sistema pode assumir qualquer um. Finalmente, em autômatos com movimento vazio, a transição pode assumir um conjunto de estados, isto é, autômatos com movimentos vazios também são não determinísticos.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Introdução à Teoria de Autômatos, Linguagens e Computação

HOPCROFT, J. E.; ULLMAN, J. D.; MOTWANI, R. Introdução à teoria de autômatos, linguagens e computação. Rio de Janeiro: Campus, 2002.

Linguagens Formais e Autômatos

MENEZES, P. B. Linguagens formais e autômatos. 6. ed. São Paulo: Bookman, 2011.

Theory of Computation

SIPSER, M. Theory of computation. India: [s.n.], 2008.

Introdução à teoria da computação

_____. Introdução à teoria da computação. 2. ed. São Paulo: Thompson Learning, 2007.

Referências

GERSTING, J. **Fundamentos matemáticos para a Ciência da Computação**. 5. ed. Rio de Janeiro: LTC, 2004.

HOPCROFT, J. E. ***Introduction to automata theory, languages and computation.*** Massachusetts, USA: Addison-Wesley, 1979.

_____.; ULLMAN, J. D.; MOTWANI, R. **Introdução à teoria de autômatos, linguagens e computação**. Rio de Janeiro: Campus, 2002.

MENEZES, P. B. **Linguagens formais e autômatos.** 6. ed. São Paulo: Bookman, 2011.

PAPADIMITRIOU, C. H.; LEWIS, H. R. **Elementos da teoria da computação**. 2. ed. Porto Alegre, RS: Bookman, 2000.

RAMOS, M. V. M. **Linguagens formais e autômatos**. [S.l.]: Universidade Federal do Vale do São Francisco, 2008. Disponível em: <http://docs.fct.unesp.br/docentes/dmec/olivete/lfa/arquivos/Apostila.pdf>. Acesso em: 27 dez. 2018.

_____.; VEGA, I. S.; JOSE NETO, J. **Linguagens formais:** teoria, modelagem e implementação. Porto Alegre, RS: Bookman, 2009.

SIPSER, M. **Introdução à teoria da computação**. 2. ed. São Paulo: Thompson Learning, 2007.

VIEIRA, N. J. **Introdução aos fundamentos da computação:** linguagens e máquinas. São Paulo: Thompson Learning, 2006.


Cruzeiro do Sul
Educacional